

API — Basic Concept

♦ API কী?

API (Application Programming Interface) হলো এমন একটি মাধ্যম/নিয়ম, যার মাধ্যমে একটি software/system অন্য একটি software/system-এর সাথে **data** আদান-প্রদান করে।

সহজভাবে:

এক **software** অন্য **software**-এর কাছে **data** চাইতে বা **data** পাঠাতে **API** ব্যবহার করে।

♦ 1. Software → Software Communication

একটি software অন্য একটি software থেকে data নিতে পারে **API**-এর মাধ্যমে।

Software A



API



Software B

Example:

আমাদের app যদি কোনো Weather Service থেকে weather data নিতে চায়:

Your App



Weather API



Weather Server



Weather Data



Your App

অর্থাৎ:

Your App → API → Other Software/Server

এখানে API হচ্ছে দুই software-এর মধ্যে যোগাযোগের মাধ্যম।

◆ 2. Frontend → Backend Communication

Web application-এ **Frontend** এবং **Backend**-ও **API**-এর মাধ্যমে **communicate** করে।

Frontend



API



Backend

ধরো React frontend-এ user একটা button চাপলো:

React Frontend



API Request



Node.js Backend



Database

Backend database থেকে data নিয়ে আবার frontend-কে পাঠায়:

Database



Backend



API



Frontend

◆ Complete Flow

একটা সাধারণ web application-এ পুরো flow:

USER



FRONTEND

(React)

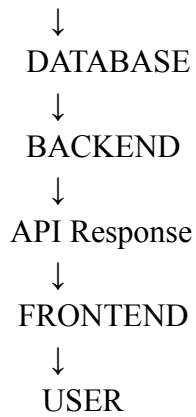


API Request



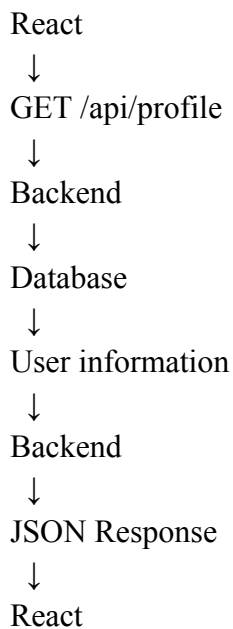
BACKEND

(Node.js)



Example:

User যদি "My Profile" button চাপ দেয়:



React তখন সেই data screen-এ দেখায়।

★ খুব গুরুত্বপূর্ণ

Frontend সাধারণত সরাসরি **database**-এর সাথে কথা বলে না।

✗ ভুল/অনিরাপদ architecture:

React → Database

✓ সাধারণ architecture:

React
↓
API
↓
Backend
↓
Database

কারণ Backend database-এর access, authentication, validation, business logic ইত্যাদি control করে।



এক লাইনে মনে রাখো

API হলো **software/system**-এর মধ্যে **communication** করার মাধ্যম, যার মাধ্যমে তারা **data request** এবং **response** আদান-প্রদান করে।

আর web development-এর ক্ষেত্রে:

Frontend → API → Backend → Database

এটাই সবচেয়ে important flow।



Shortcut

API = Software-এর মধ্যে কথা বলার মাধ্যম

Frontend → API → Backend → Database

Backend → API → Frontend

Real-life example

You → **Customer**

Restaurant kitchen → **System that prepares your food**

Waiter → **API**

You tell the waiter:

"I want a burger."

The waiter takes your request to the kitchen.

The kitchen prepares the burger.

The waiter brings the burger back to you.

CRUD

CRUD = Create, Read, Update, Delete

CRUD সাধারণত **database-এর data manage** করার জন্য ব্যবহার হয়। CRUD হলো **database-এর data** নিয়ে করা ৪টি **basic operation**।

Operation	Meaning	Example	HTTP Method
Create	নতুন data তৈরি	নতুন user যোগ করা	POST
Read	data নেওয়া/দেখা	সব user দেখা	GET
Update	existing data পরিবর্তন	user's name পরিবর্তন	PUT / PATCH
Delete	data মুছে ফেলা	user delete করা	DELETE

API-এর সাথে CRUD

ধরো, একটি **User Management System**:

React Frontend

↓

API

↓

Backend

↓

Database

Create:

POST /users

→ নতুন user database-এ যোগ হবে।

Read:

GET /users

→ database থেকে user-এর data আসবে।

Update:

PUT /users/1

→ ID 1-এর user-এর data পরিবর্তন হবে।

Delete:

DELETE /users/1

→ ID 1-এর user delete হবে।

 সবচেয়ে **important**

CRUD = Data-এর উপর কী কাজ করছি

API = সেই কাজের জন্য Frontend ↔ Backend
communication করার মাধ্যম

তাই:

Frontend API-এর মাধ্যমে Backend-কে request করে, আর Backend CRUD operation-এর মাধ্যমে Database-এর data manage করে।

API হলো **communication**-এর মাধ্যম, আর **CRUD** হলো **data**-এর ওপর কী কী **basic** কাজ করছি তার ধারণা।

Fetch data

1. প্রথমে আমরা একটা **data-fetching function** বানাই

```
const postsPromise = async () => {  
  const res = await  
  fetch("https://jsonplaceholder.typicode.com/posts");  
  const data = await res.json();  
  return data;  
}
```

এখানে কী হচ্ছে?

postsPromise()
↓
fetch()
↓
Server-এ request
↓
Response আসে
↓
res.json()
↓
Actual data পাওয়া যায়
↓
return data

2. **async** কী?

যে function-এর ভিতরে **await** ব্যবহার করবে, সেই function-কে সাধারণত **async** করতে হয়।

```
async function getUsers() {  
  const response = await fetch("/api/users");  
}
```

এখানে:

async → function-কে asynchronous করে

await → Promise resolve হওয়া পর্যন্ত অপেক্ষা করে

2. **await** কী করছে?

এটা সবচেয়ে important।

```
const res = await  
fetch("https://jsonplaceholder.typicode.com/posts");
```

Process:

fetch("/api/posts")

↓

Promise

↓

await

↓

Promise complete হওয়ার অপেক্ষা

↓

response পাওয়া

তারপর JSON data নিতে:

```
const data = await response.json();
```

3. তাহলে **Component** কেন লাগছে?

কারণ **function** বানানো আর **function** চালানো এক জিনিস না।

তুমি শুধু এটা লিখেছ:

```
const postsPromise = async () => {  
  ...  
}
```

এখানে তুমি শুধু **function** তৈরি করেছো।

এখনও function execute হয়নি।

যেমন:

```
const add = () => {  
  return 10 + 20;  
}
```

এখানে **add** function তৈরি হয়েছে।

কিন্তু:

`add()`

না লিখলে **10 + 20** calculate হবে না।

একইভাবে:

`postsPromise`

মানে function-টাকে reference করা।

কিন্তু:

`postsPromise()`

মানে function-টা **execute** করা এবং fetch শুরু করা।

4. তাহলে **Suspense** কেন?

এখন ধরো data আসতে ২ **seconds** লাগছে।

তখন React-এর প্রশ্ন:

"Data আসার আগ পর্যন্ত screen-এ কী দেখাব?"

এই জায়গায় **Suspense** কাজে আসে।

Concept:

Component

↓

Data দরকার

↓

Data এখনও আসেনি

↓

Suspense

↓
Loading UI দেখাও
↓
Data এসে গেছে
↓
Actual UI দেখাও

Suspense = waiting-এর system
fallback = waiting-এর সময় কী দেখাবে

Example

```
<Suspense fallback={<p>Please wait...</p>}>  
  <Posts props = {postsPromise()}></Posts>  
</Suspense>
```

এখানে:

Suspense
|
├── fallback = <p>Loading...</p>
└── Posts

যখন **Posts** ready না:

Suspense বুঝলো:
"Posts এখনো ready না"

↓
fallback দেখাও

↓
Loading...

যখন **Posts** ready:

UserCard ready

↓

fallback সরিয়ে দাও



UserCard দেখাও

তাহলে আলাদা করে:

Suspense

এটা **React component/feature**:

```
<Suspense>
```

...

```
</Suspense>
```

এর কাজ:

Child component-এর কোনো কিছু ready হওয়ার জন্য অপেক্ষা করা এবং সেই waiting state handle করা।

fallback

এটা **Suspense**-এর একটি **prop**:

```
fallback={<Loading />}
```

এর কাজ:

Child ready না হওয়া পর্যন্ত কী **UI** দেখাবে সেটা বলা।

পুরো process একসাথে

1. postsPromise function তৈরি



2. Component থেকে postsPromise() call

- ↓
3. fetch() server-এ request পাঠায়
 - ↓
 4. Promise pending থাকে
 - ↓
 5. Suspense → "Loading..." দেখায়
 - ↓
 6. Server response দেয়
 - ↓
 7. res.json() → actual data
 - ↓
 8. Component data নিয়ে UI render করে

সবচেয়ে **important distinction:**

async/await = asynchronous data fetching handle করার জন্য

Component = সেই function call করে data ব্যবহার করার জায়গা

Suspense = data ready না হওয়া পর্যন্ত **fallback UI** দেখানোর **mechanism**

5. Promise pending মানে হলো:

কাজটা শুরু হয়েছে, কিন্তু এখনো **result** পাওয়া যায়নি।

example দিয়ে দেখি:

```
const postsPromise = async () => {
  const res = await
fetch("https://jsonplaceholder.typicode.com/posts");
  const data = await res.json();
  return data;
}
```

যখন:

`postsPromise()` call করে, `async function` call করলে JavaScript সাথে সাথে একটি **Promise-object** return করে। তাই `postsPromise()` call করলে component একটি pending Promise-object পায়।

এরপর `fetch()` server/API-তে request পাঠায়। Response আসতে কিছুটা সময় লাগতে পারে। এই সময় Promise-object pending অবস্থায় থাকে।

যখন component `use(...)` দিয়ে ওই pending Promise-object-টি read করে, React বুঝতে পারে যে data এখনো ready হয়নি। তখন component **suspend** করে, এবং কাছের **Suspense** boundary-এর **fallback** UI দেখায়।

যখন `fetch()` শেষ হয়, আগে যে **Promise-object** টি **component**-এ পাঠানো হয়েছিল, সেই একই Promise-object-এর সাথে `fetch()` থেকে পাওয়া result/data **automatically connected** হয়ে যায়। `postsPromise()` নতুন করে কোনো data component-এ পাঠায় না।

এরপর `use()` ওই একই Promise-object-টি read করে এবং Promise-object-এর সাথে connected হওয়া resolved data বের করে নেয়। React তখন component-টিকে আবার render করে এবং **Suspense**-এর fallback সরিয়ে actual **Posts** দেখায়।

সবচেয়ে গুরুত্বপূর্ণ বিষয়: `postsPromise` প্রথমে শুধু একটি Promise হিসেবে component-এ যায়। `fetch()` শেষ হওয়ার পর সেই Promise-এর **resolved result** হিসেবে **data automatically connected/associated** হয়ে যায়। Promise function দ্বিতীয়বার **component**-এ **data** পাঠায় না। `use()` ওই একই Promise থেকে resolved data বের করে নেয়।

Promise-object আগে পাঠানো হলো

↓

`fetch` শেষ হলো

↓

আগের সেই Promise-object-এর সাথে data/result connected হলো

↓

`use(Promise)`

↓

`use()` সেই data বের করে দিল

App component

|

| `postsPromise()`

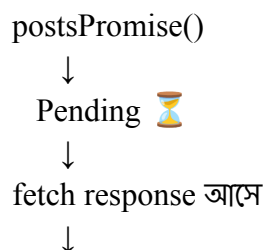


Important: শুধু Promise Posts-এ pass করলেই Suspense fallback দেখাবে না। Posts-এর ভিতরে use(props) pending Promise read করলে তখন component suspend করবে, আর Suspense fallback দেখাবে।

Promise-এর ৩টা possible state:

Pending → এখনো result আসেনি ⌚
Fulfilled → সফলভাবে result এসেছে ✓
Rejected → error হয়েছে ✗

তোমার ক্ষেত্রে:



json() হয়
↓
Fulfilled ✓
↓
Actual posts data

একটা analogy:

তুমি restaurant-এ খাবার order দিলে waiter একটা **token** দিল।

Token = Promise

খাবার আসার আগে = Pending ⌚

খাবার আসার পরে = Fulfilled ✓

6. Re-render মানে কি :

সহজভাবে বললে **re-render** মানে **component**-এর **render logic** আবার **run** করা। কিন্তু পুরো application আবার শুরু থেকে run করা না।